



ECOLE TECHNIQUE
ECOLE DES MÉTIERS LAUSANNE

« TPI N°2165 - ARCALUDO »

Application mobile de gestion d'une bibliothèque
personnelle de jeux vidéo

CFC Informatique – CID4B



04 MAI – 01 JUIN 2026

Candidat : Adrian Federico Toledo Campoverde

Chef du projet : Pascal Piot

Expert 1 : Pantic Nemanja

Expert 2 : Ernesto Montemayor

Version : 0.1.0

Table des matières

Table des illustrations	3
Gestion du document	4
Historique des versions	4
Abréviations et glossaire	4
1. Rapport de gestion et de réalisation	5
1.1. Contexte et mandat	5
1.2. Méthode et organisation	7
1.3. Analyse des exigences.....	11
1.4. Gestion des risques et mesures.....	12
1.5. Conception.....	13
1.6. Planification et suivi	33
1.7. Réalisation	35
1.8. Tests et validation	35
1.9. Livraison	35
1.10. Conclusion et bilan personnel.....	35
2. Documentation du produit	35
2.1. Prérequis et environnement cible	35
2.2. Installation et déploiement	35
2.3. Guide d'utilisation	35
2.4. Exploitation et maintenance	35
2.5. Architecture technique.....	35
3. Résumé	35
3.1. Contexte et mandat	35
3.2. Solution réalisée	35
3.3. Résultats principaux.....	35
3.4. Limites et suites.....	35
4. Annexes	35
4.1. Journal de travail.....	35
4.2. Bibliographie et références.....	35
4.3. Liste des livrables remis	35

Table des illustrations

Gestion du document

Historique des versions

Version	Date	Modifications

Abréviations et glossaire

1. Rapport de gestion et de réalisation

1.1. Contexte et mandat

Depuis l'apparition des jeux vidéo aux années 70's, acheter, jouer et collectionner des jeux a été une grande passion pour de nombreux joueurs. Au fil du temps, les jeux vidéo et leur industrie ont évolué de manière significative, atteignant des niveaux économiques comparables à ceux du cinéma ou d'autres loisirs.

Aujourd'hui, des sorties de jeux, qu'elles soient numériques ou physiques, ont lieu chaque semaine et même chaque jour, sur consoles, ordinateurs, plateformes numériques et aussi sur mobile. Cette abondance de sorties rend de plus en plus difficile le suivi de nos propres jeux et complique être à jour sur le marché.

Dans ce contexte social, ce projet de TPI a pour objectif d'offrir un outil proche des utilisateurs, leur permettant de rassembler l'ensemble de leur collection de jeux vidéo et d'assurer le suivi de leurs jeux joués, achetés ou souhaités.

Pour cela, dans ce rapport nous présentons l'application Arcaludo : une application mobile Android qui, grâce à l'utilisation d'une bibliothèque de jeux externe à jour, offre à l'utilisateur la possibilité d'enregistrer sur son appareil mobile sa collection des jeux vidéo multiplateforme. De plus, l'application fournit des statistiques liées au profil utilisateur et permet, le cas échéant, de les partager avec la communauté.

Sur le marché des applications mobiles, il existe déjà des solutions proposant des services similaires, mais elles ne correspondent pas exactement aux caractéristiques définies pour ce projet. Par exemple, CLZ Games est dédié à la collection physique de jeux, GameTrack se concentre sur le suivi de progression, et GG App met l'accent sur la communauté. Arcaludo vise à combiner ces fonctions au sein d'une plateforme conçue dans le cadre d'un projet étudiant de fin d'études en informatique CFC, spécialité développement d'applications.

1.1.1. Périmètre

Pour préciser le périmètre de l'application et indiquer les éléments exclus du projet, voici une table de périmètre (in-scope / out-of-scope) :

Inclus (In-Scope)	Exclus (Out-of-scope)
Application mobile Android	Version iOS
Pages : collection, détail du jeu, état (profil et jeu)	Synchronisation cloud
Intégration de l'API externe RAWG.io pour la recherche de jeux	Intégration réelle des comptes de plateformes (Steam, Xbox, Epic, PSN...)
Authentification JWT et gestion des profils	Connexion hors ligne (se connecter sans internet)
CRUD sur la collection de jeux et filtres	Notifications push
Stats par jeu (acquis, en cours, terminé et souhaité)	Système de recommandations
Supplémentaire : statistiques de profil selon la collection et page communauté pour partager les statistiques	Système de messagerie entre utilisateurs

1.1.2. Objectifs SMART

Précisant les objectifs selon la méthode SMART, nous pouvons spécifier, mesurer et connaître l'étendue et la pertinence des objectifs dans le cadre temporel du projet. De plus, cela permet de définir des critères d'acceptation vérifiables, présentés dans le tableau suivant :

Objectifs SMART	Détails	Critères d'acceptation
Gestion de collection	Les opérations CRUD fonctionnent correctement avec l'API et MySQL	On peut ajouter des jeux depuis l'API. Les jeux peuvent être édités et supprimés de la liste. Les informations affichées (titre, plateforme, détails, image) sont mises à jour et des notifications d'ajout/modification/suppression sont émises ; commentaires, évaluations et états sont pris en charge.
Recherche et import depuis l'API	Rechercher et importer des jeux par informations ou catégorie	Les jeux s'affichent selon les critères de recherche (titre) ou les filtres (plateformes, genres). Une notification est affichée si aucune correspondance n'est trouvée.
États et statistiques	États pour jeux : acquis, en cours (playing), terminé, souhaité	Ces états se mettent à jour dans la base de données et dans l'interface. Les nombres et pourcentages correspondants sont affichés.
Authentification et paramètres	Enregistrement et connexion avec session persistante. Accès optionnel à la communauté	Validation des formulaires ; la session persiste jusqu'au logout ; l'accès aux actions protégées est restreint.
Navigation et interface UI/UX	Implémenter les écrans principaux, navigation cohérente et fluide	Écrans : ma liste, détail, recherche/import depuis la liste générale, profil/auth et communauté.
Tests et qualité	Tests unitaires et d'intégration	Tests unitaires pour les services API ; scénarios E2E documentés.
Documentation et livraison	Document complet et préparation d'une démonstration	Exécution vérifiée sur émulateur et appareil pour la démo ; documentation PDF complète incluse.

1.1.3. Contenu et livrables

Durant la rédaction de ce rapport, seront détaillées l'élaboration du projet depuis les premiers raisonnements, les recherches et analyses, la méthodologie utilisée, les concepts, les designs, la planification, le développement, la réalisation, le déploiement et les conclusions, ainsi que les éléments annexes nécessaires au résultat de ce TPI. L'objectif est de présenter au responsable de projet et aux experts

une application fonctionnelle via une démonstration, documentée dans un rapport correspondant à un total de 90 heures de travail, en respectant, si nécessaire, les lois fédérales sur la protection des données et le cadre étudiant de l'école ETML.

Livrables prévus pour ce projet :

- Dépôt Git publique avec l'arborescence
- Application mobil MAUI fonctionnelle
- Code source du frontend y backend
- Rapport de projet
- Planification
- Journal de travail

1.2. Méthode et organisation

1.2.1. Méthodologie choisie

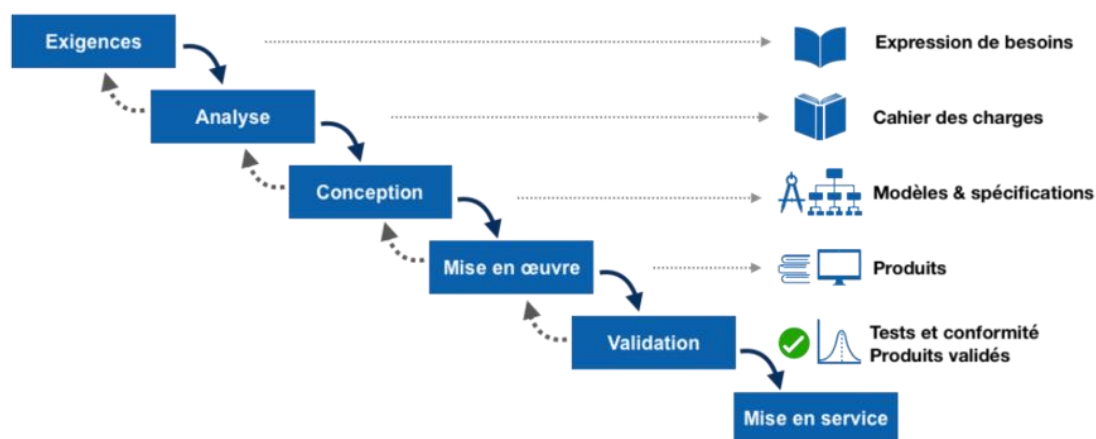
Pour la réalisation de ce projet, différentes méthodologies de travail ont été évaluées (exemple : agile ou cycle en V). En comparant ces méthodologies et compte tenu du contexte et du temps disponible, j'ai été déterminé que la méthode Waterfall est la plus appropriée.

Le choix principal en faveur du Waterfall repose sur le fait que le projet part d'un CDC où les exigences sont définies, grâce à Git, au journal de travail et au rapport, le projet peut être tracé. Ainsi, le travail peut être réparti selon les phases du Waterfall pour respecter le délai de livraison. Par exemple, si l'on suivait la méthodologie agile, le travail en sprints, les démonstrations et les adaptations constantes seraient défavorables au regard du nombre d'heures disponibles pour le développement, et impliqueraient des réunions fréquentes pour définir l'évolution du projet. Pour respecter le délai, rester autonome et disposer d'une planification et d'une documentation complètes, Waterfall apparaît comme un outil adapté.

Arcaludo avec Waterfall :

- Première phase – Exigences : le CDC décrit les besoins et objectifs de l'application ; le travail consiste à relier ces demandes et à adapter le CDC au projet.
- Deuxième phase – Analyse : cette étape sert à définir comment le projet sera investigué et à identifier les points clés pour choisir les concepts. Cette phase et avec les exigences c'est estimé à environ 20 % du temps total.
- Troisième phase – Conception : cette étape établit la feuille de route pour le développement : modèles, diagrammes, structures, spécifications et maquettes doivent être conçus et réalisés afin de préparer la mise en œuvre.
- Quatrième phase – Mise en œuvre : en suivant les définitions de la conception, cette étape consiste à développer l'application en intégrant les éléments étudiés précédemment. Le résultat attendu est un produit de base, l'application Arcaludo. Les phases de conception et de mise en œuvre sont considérées comme l'implémentation et représentent environ 60 % du temps.
- Cinquième phase – Validation : par le biais de différents types de tests, l'application doit être validée et vérifiée par rapport aux exigences définies. Les tests représentent environ 10 % du temps de travail.

- Sixième phase – Mise en service : dans le cadre scolaire, il n'est pas nécessaire de publier l'application sur une boutique, mais elle doit fonctionner pour permettre une démonstration. Outre la démo, la documentation finale est attendue ; la rédaction du rapport représente environ 10 % des 90 heures.



1.2.2. Outils de gestion

Et de la même manière qu'une méthodologie est établie pour le projet, il convient également de choisir les outils de gestion destinés à l'élaboration de ce travail :

Gestion	Outil
Planification	Excel – Gantt - Trello
Documentation	Word - Rapport
Versioning	Git
Suivi	Journal de travail - Excel
Architectures techniques	Looping + draw.io
API	RAWG
Maquettes	Figma
Test API	Thunder Client
Conteneurisation	Docker Desktop
IA assistance	Claude et Copilot
Communications	Email Outlook

1.2.3. Outils de développement

Actuellement, plusieurs services API sont disponibles sur Internet, mais pour ce projet scolaire, il a fallu rechercher une API présentant les caractéristiques suivantes : accès gratuit, large choix de jeux vidéo modernes issus de différentes plateformes, bonne disponibilité de l'infrastructure, simplicité d'utilisation et respect des délais impartis pour le projet. À cette fin, des informations ont été recueillies sur différentes API répondant à ces critères.

API	Jeux	Auth	Gratuit	Pros	Contre
RAWG.io	500'000 +	API Key	Limité	Plan gratuit limité à 50'000 requêtes/mois suffisant.	Problèmes d'instabilité.
IGDB (Twitch API)	230'000 +	OAuth2 Twitch	Oui	Données complètes	Complexité liée à l'intégrité et Token à actualiser toutes les 2 heures
FreeToGame	400+	-	Oui	Facile à intégrer sans besoin d'autorisation.	Uniquement des jeux de type Free-To-Play
GiantBomb	65'000+	API Key	Limité	Informations détaillées pour les éditeurs	Licence gratuite très limitée et manque d'information technique
MobyGames	300'000 +	API Key	Limité	Contenu de jeux rétro	Catalogue non mis à jour

Cette comparaison nous permet de constater que RAWG.io offre les meilleurs résultats pour ce projet. Son vaste catalogue et sa version gratuite adaptée à mes besoins en font l'outil de prédilection pour l'application mobile, même s'il est parfois en maintenance et indisponible pour d'éventuels nouveaux jeux.

D'autre part, IGDB a été écarté en raison de sa complexité. Malgré sa bonne disponibilité, la sécurité élevée requise pour l'intégrer à l'infrastructure allonge le temps de travail, car il nécessite un compte Twitch, une autorisation OAuth2 avec demande de Client ID + Client Secret et une mise à jour constante du Token.

Les autres options sont insuffisantes en termes de contenu, proposent un contenu différent de celui que je souhaite afficher ou, tout simplement, leur version gratuite ne répond pas à mes besoins.

1.2.4. Titre

Lors de la sélection du titre de cette application, j'ai voulu mettre en avant plusieurs aspects à différencier comme qu'il ne soit pas en rapport avec d'autres applications, marques ou sites web. Un autre point important est la linguistique : comme il s'agit d'un projet d'origine suisse francophone, il est important qu'il soit compréhensible dans la langue, et un plus s'il est international. En suivant ces critères, j'ai établi cette liste de toutes les idées qui représentent la fonctionnalité de l'application, avec leurs avantages et leurs inconvénients.

Titre	Disponibilité	Pros	Contres
MyGamesPark	Déjà utilisé avec GamesPark	Nom sympathique	Peu professionnel, ne suggère pas

			une collection et en anglais
MyGameChest	Quelques sites similaires	Coffre bonne métaphore pour collection	Moins adapté à un public francophone
MyGamesCollection	App existe	Très clair et explicite	Trop long et en anglais
MyGameLibrary	Quelques sites similaires	Très clair et explicite	Trop long et en anglais
PixelCoffre	Disponible	Métaphore clair	« Coffre » uniquement français
PixelZone	Déjà utilisé avec sites, jeux et projets existants	Court et moderne	Nom déjà pris
MesPixels	Disponible	Personnel avec « Mes »	Peut être confondu avec Google Pixel ou assets graphiques
PixelScore	Déjà pris.	Court et international pour l'anglais et français. Il y a Référence aux statistiques	Ne dit pas que c'est une collection directement
Arcaludo	Disponible	« Arca » du latin pour définir une boîte ou coffre. Et « ludo » du latin pour définir jeu.	Si on ne connaît pas le latin, on ne comprend pas son signifié.

Après avoir analysé les avantages et les inconvénients de chaque nom, j'ai retenu «Arcaludo» comme premier choix. Arcaludo est la combinaison de deux mots latins désignant un coffre à jeux. D'origine latine, ce nom est facile à prononcer dans le monde entier d'origine latin (en anglais, en français ou en espagnol). Bien que le mot n'ait pas d'impact direct lié à sa signification, contrairement aux autres, il s'agit d'un nom propre, ce qui évite d'éventuelles similitudes avec d'autres applications ou sites web. D'autre part, « arca » est proche de « arcade » et « ludo » de « ludothèque », de sorte qu'il présente une proximité avec les jeux vidéo, même sans connaître le latin. Et le dernier aspect important est que la recherche par ordre alphabétique fait apparaître l'application en premier.

1.3. Analyse des exigences coffre

- Exigences fonctionnelles

Exigences fonctionnelles	Priorité
L'utilisateur peut créer un compte avec email, nom utilisateur et mot de passe	P1
L'utilisateur peut se connecter avec authentification JWT et déconnecter	P1
L'utilisateur peut rechercher des jeux via l'API RAWG.io	P1
L'utilisateur peut consulter la fiche détaillée d'un jeu	P1
L'utilisateur peut ajouter un jeu à sa collection	P1
L'utilisateur peut supprimer un jeu de sa collection	P1
L'utilisateur peut modifier le statut (acquis/en cours/terminé/ souhaits)	P1
L'utilisateur peut modifier les informations extra (note, commentaire, temps de jeu, plateformes possédées)	P1
L'utilisateur peut consulter la bibliothèque, wishlist et collection complète	P1
L'utilisateur peut filtrer ses jeux par derniers ajouts	P1
L'utilisateur peut trier ses jeux par nom A-Z et Z-A	P1
L'utilisateur peut filtrer ses jeux par statuts	P1
L'utilisateur peut chercher ses jeux par barre de texte de son titre	P1
L'utilisateur peut filtrer ses jeux par note metacritic	P2
L'utilisateur peut filtrer ses jeux par date de lancement	P2
L'utilisateur visualise ses statistiques personnelles (compteurs en cours/terminés/ souhaits)	P2
L'utilisateur peut supprimer son compte et toutes ses données	P2
L'utilisateur peut filtrer ses jeux par plateforme	P3
L'utilisateur peut filtrer ses jeux par catégorie	P3
L'utilisateur a la possibilité de consulter son droit et ses conditions d'utilisation des données	P3
L'utilisateur peut activer/désactiver sa participation à la Communauté	P3
L'utilisateur peut utiliser sur les jeux les déplacements horizontaux pour ajouter au catalogue ou les supprimer	P3
L'utilisateur peut voir les compteurs des autres utilisateurs à communauté	P3

- Exigences non fonctionnelles

Exigences non fonctionnelles	Priorité
Qualité : Code organisé selon le modèle MVVM et vérifié par Tests	P1
Sécurité : Respecter confidentialité et stockage local des identifiants (hachés bcrypt et par la loi suisse)	P1
Robustesse : L'application et l'API en affichant des messages en cas de validation ou problèmes.	P1
UX : Navigation cohérente et écrans principaux accessibles	P1
Performances : La recherche via l'API doit répondre rapidement et sans boucles	P2
Environnement : utilisation de docker pour faciliter l'intégration de backend et compatible Android	P2
Tests : les tests sont valides et documentés	P2
UX : Il y a suffisamment d'espace pour éviter les textes trop longs	P3

1.4. Gestion des risques et mesures

Points forts : Combine collection digital, suivi, communauté, intégration API, architecture MVVM, authentification de base.	Faiblesses : limitée à 90 h, dépendance d'une API externe, disponible que sur Android
Opportunités : renforcement sur la communauté, possibilité de partager photos avec les éditions spéciales o contenu uniques. Fiabilité sur les avis	Menaces : Concurrents bien établis, changements dans les API ou les limites, exigences légales/confidentialité.

Risques organisationnels

Risque	Impact	Mesure à résoudre
Temps insuffisant (90 h)	Élevé	Prioriser les fonctionnalités principales pour avoir un MVP. Découper tâches et documenter avancement.
Dépendance à une API externe (RAWG)	Élevé	Prévenir et simuler erreurs d'API, planification de tests sur l'API, connaître les limites d'utilisation de l'API
Gestion de données personnelles et consentement	Élevé	Recherche d'information sur les lois suisses et les données et l'adapter a mes besoins.
Problèmes d'intégration d'environnement	Faible	Docker-compose stable avec documentation des commandes d'exécution. Intégration de backend et frontend comment priorité.
Tests insuffisants	Moyen	Définir scénarios et tests bien détaillés sur le planning de test. Prévenir et justifier chaque point.

1.5. Conception

1.5.1. Architecture technique

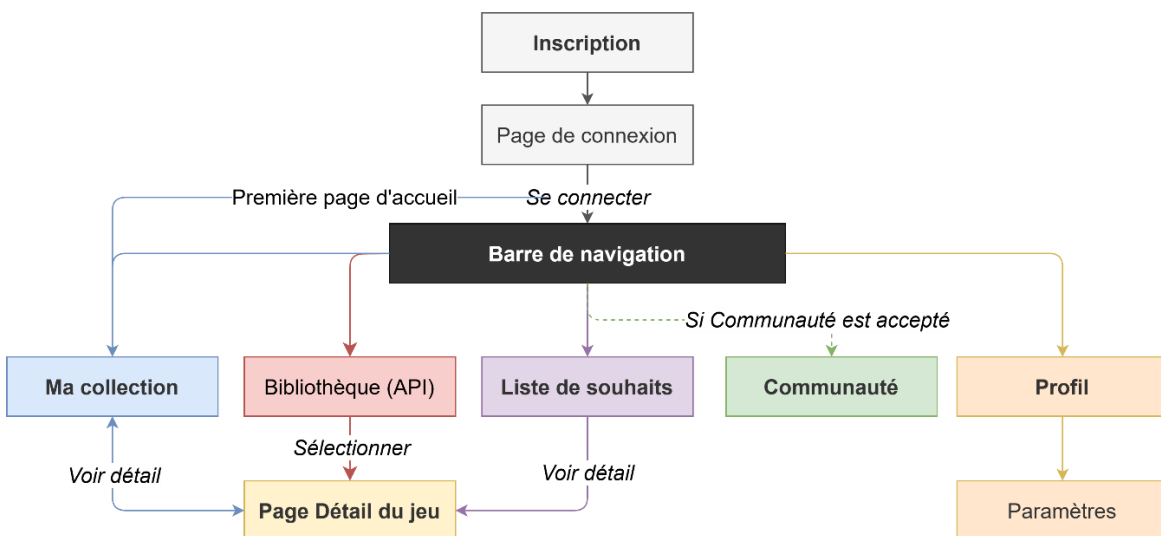
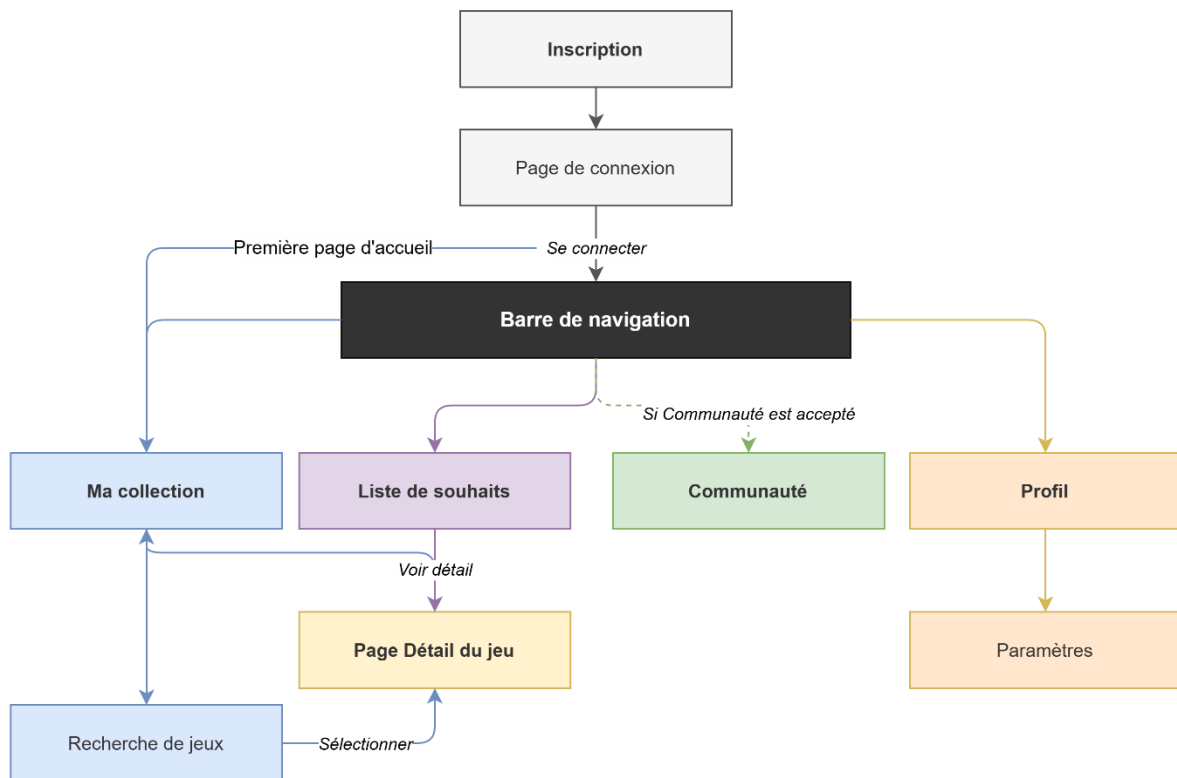


L'architecture technique d'Arcaludo est construit sur quatre composants principaux qui fonctionnent ensemble : une application mobile Android développée en .NET MAUI, une API REST Node.js/Express, une base de données MySQL, et un environnement Docker. Ces technologies ont été retenus car ce sont des technologies enseignées et pratiquées à l'ETML durant la formation CFC

MySQL convient au modèle relationnel du projet et permet de gérer proprement les relations et données nécessaires. Node.js et Express facilitent la création d'une API REST structurée selon le modèle MVC. NET 9 MAUI permet de développer une application mobile Android native en C# avec le patron MVVM. Enfin, Docker simplifie l'environnement de développement en regroupant MySQL, phpMyAdmin et le backend dans un ensemble et à la gestion des variables d'environnement.

1.5.2. Flux de navigation

Pour le flux de navigation, deux possibilités ont été envisagées, qui conservent la structure de base, mais modifient la manière de rechercher et d'ajouter un jeu depuis la bibliothèque de l'API. Pour ce premier schéma, le système de flux est vraiment résumé aux seules pages d'accès. Les actions possibles de l'utilisateur seront détaillées par la suite.



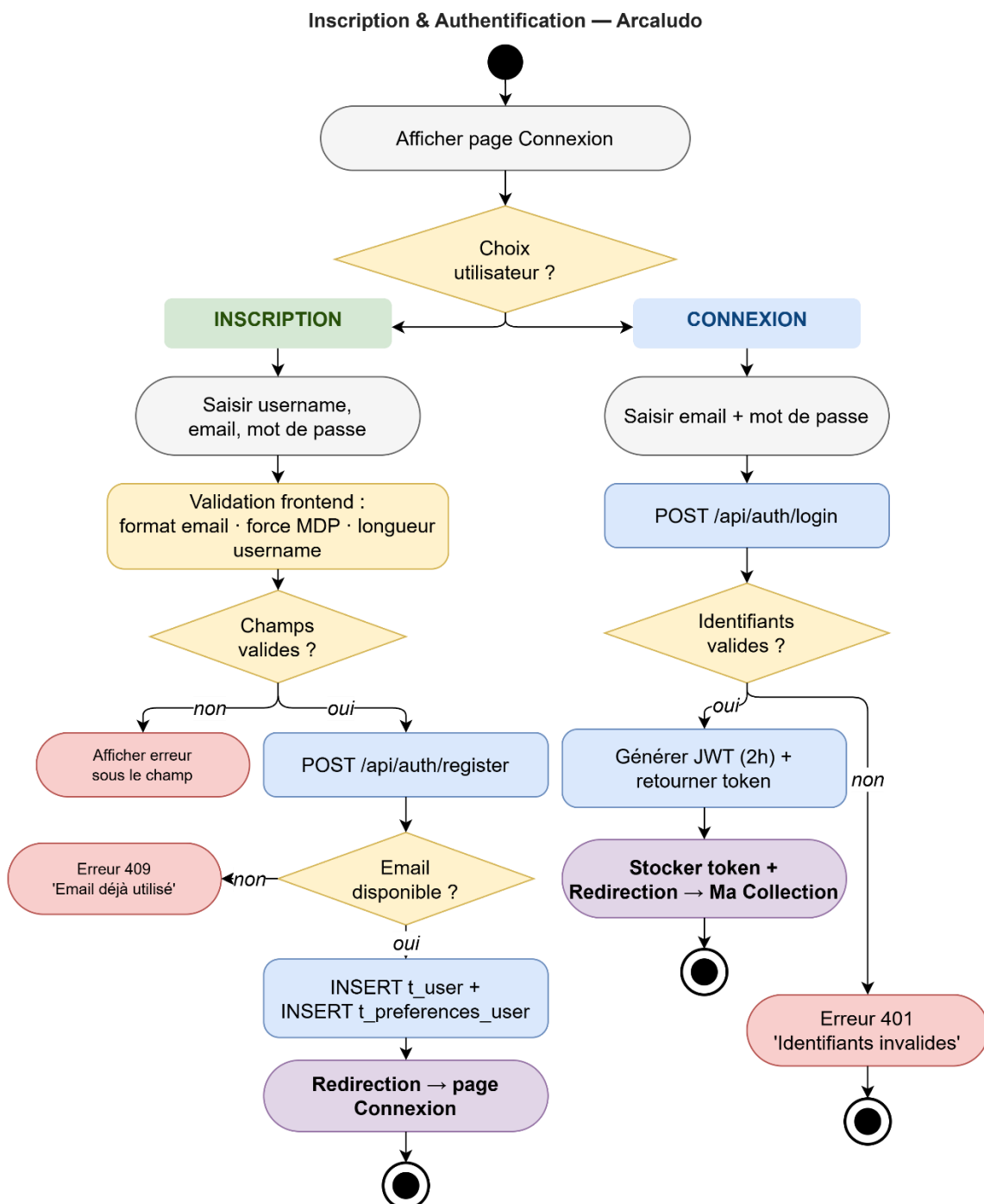
Pour commencer, l'utilisateur devra s'inscrire afin de vérifier son compte. Une fois son compte enregistré, il pourra se connecter et accéder à l'application.

Après la connexion, la barre de navigation de l'application s'affiche, avec la page « Ma collection » en premier. D'autres pages sont disponibles : la liste des favoris, la page de la communauté, si les droits d'utilisation des données ont été acceptés, et le profil avec les paramètres de l'utilisateur. Depuis la collection et la liste de souhaits, il sera possible de sélectionner un jeu pour afficher ses détails.

Pour le système d'ajout de jeu, après discussion avec le chef d'équipe, nous avons vu que le premier schéma, qui consiste à accéder à « ma collection » pour effectuer

une recherche de jeu dans l'API, impliquerait la création d'une sous-page. Cependant, pour éviter la création de sous-pages, nous avons vu qu'il serait plus commode et satisfaisant pour le client de créer une page à part destinée à la liste complète des jeux de l'API, car il pourrait ainsi voir directement les nouveaux jeux ou explorer la bibliothèque des jeux disponibles.

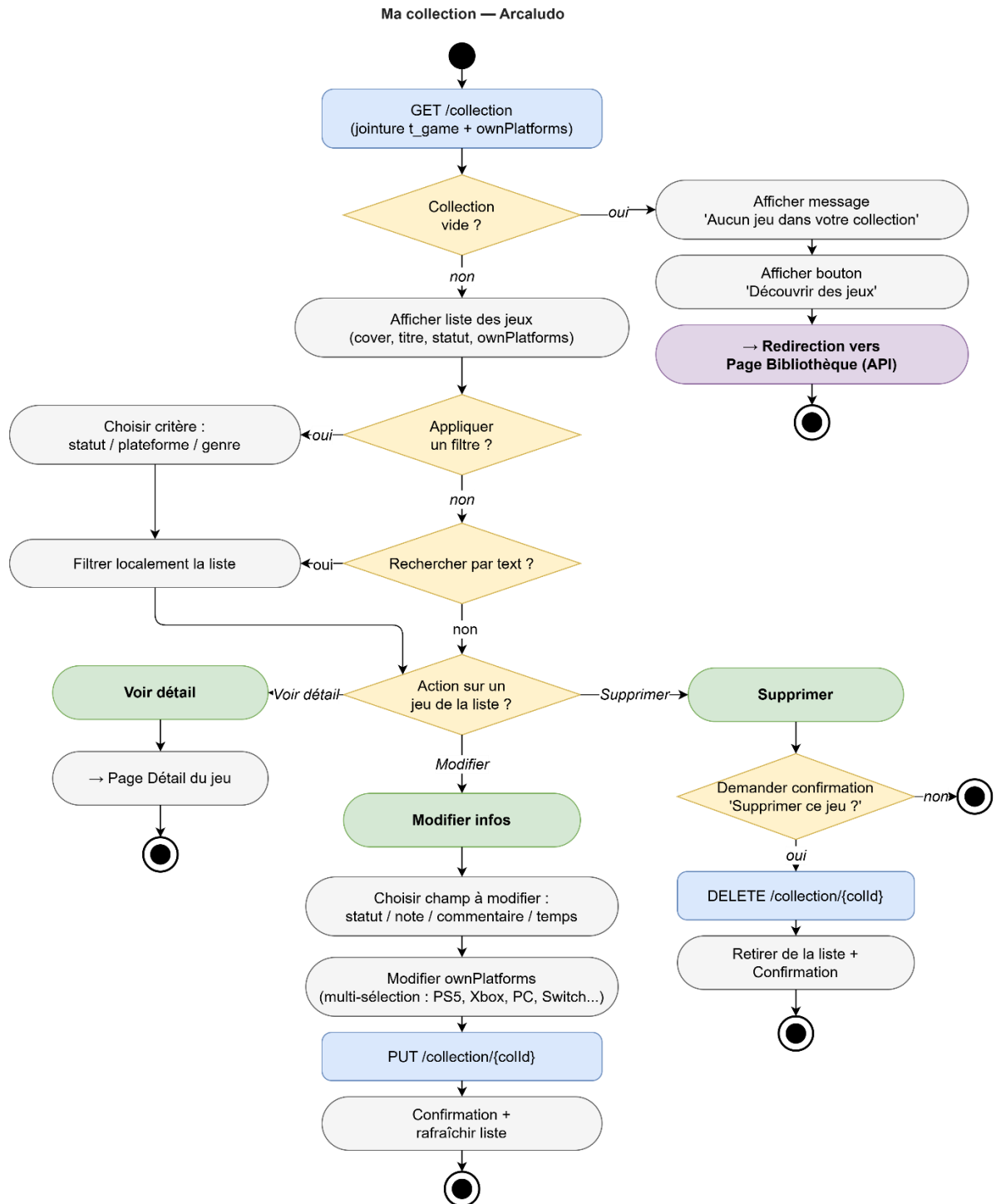
1.5.2.1. Inscription et authentification



Ce diagramme montre les deux flux de connexion de l'application. La branche gauche, l'inscription, détaille la validation des champs puis l'appel 'POST

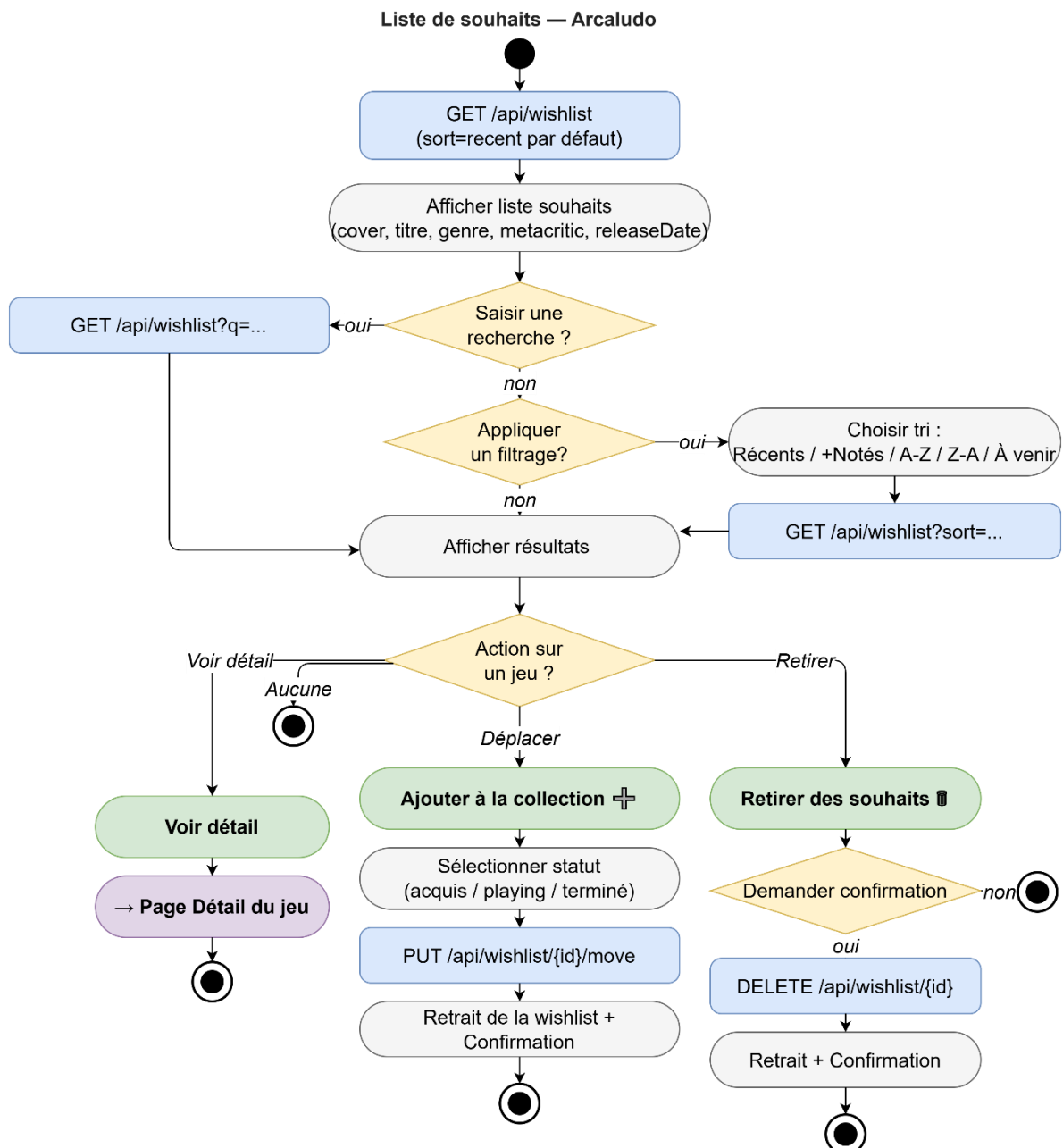
/api/auth/register` qui vérifie la disponibilité de l'email et crée automatiquement ses préférences par défaut. La branche droite, connexion, représente l'appel `POST /api/auth/login`, la vérification bcrypt du mot de passe et la génération du token JWT stocké localement sur l'appareil, suivi de la redirection vers la page principal Ma Collection.

1.5.2.2. Ma collection



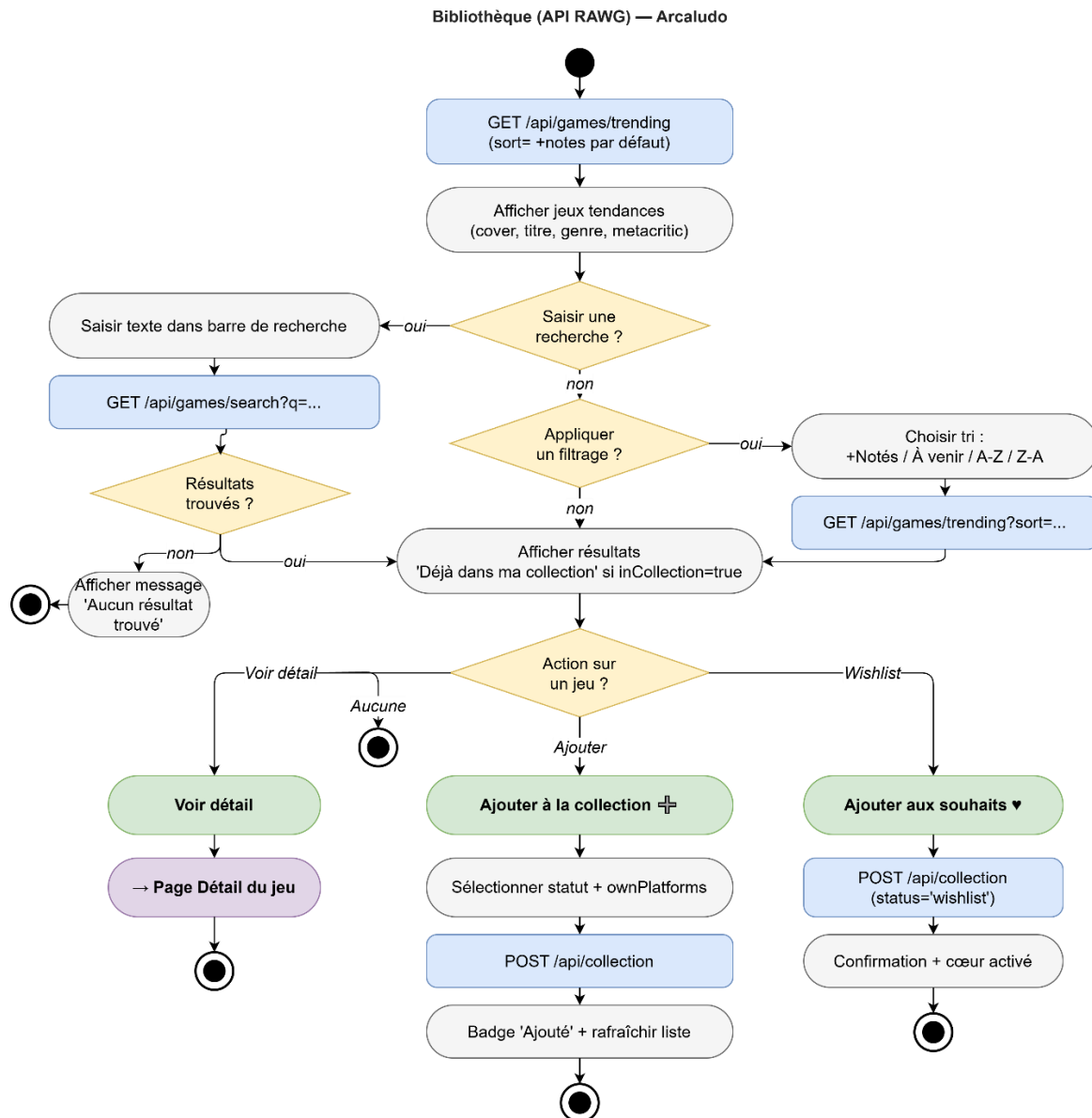
La page d'accueil, Ma collection, affiche les jeux de l'utilisateur acquis. S'il n'en a aucun, il est envoyé vers la Bibliothèque. Sinon, il peut voir un jeu, modifier ses infos ou le supprimer. Des filtres et un tri A-Z sont disponibles.

1.5.2.3. Page Wishlist



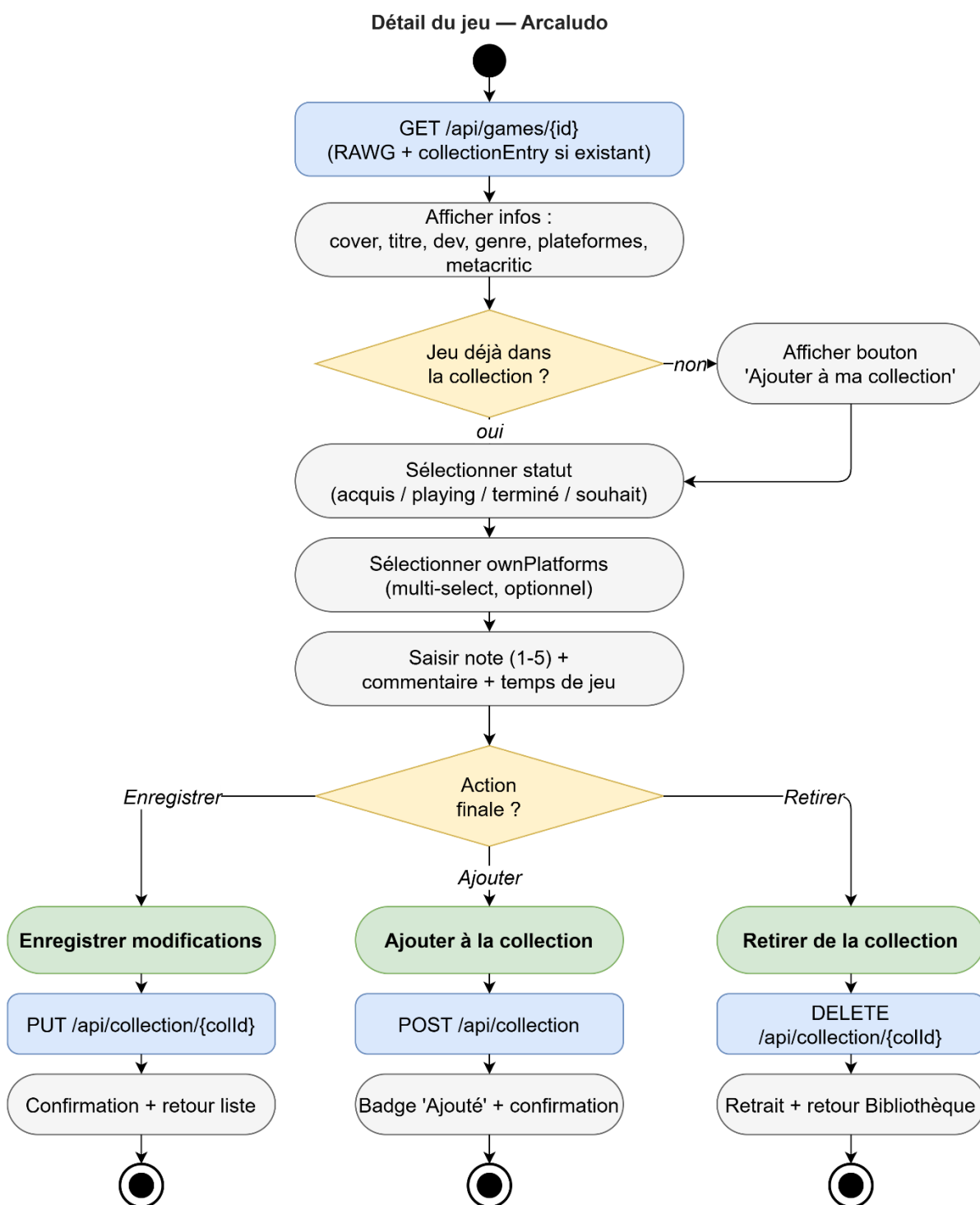
La Wishlist affiche les jeux marqués comme souhaités, triés du plus récent ajoutés au plus ancien, et aussi les plus notes ou recherche par text. L'utilisateur peut les déplacer vers sa collection ou les retirer.

1.5.2.4. Page Bibliothèque



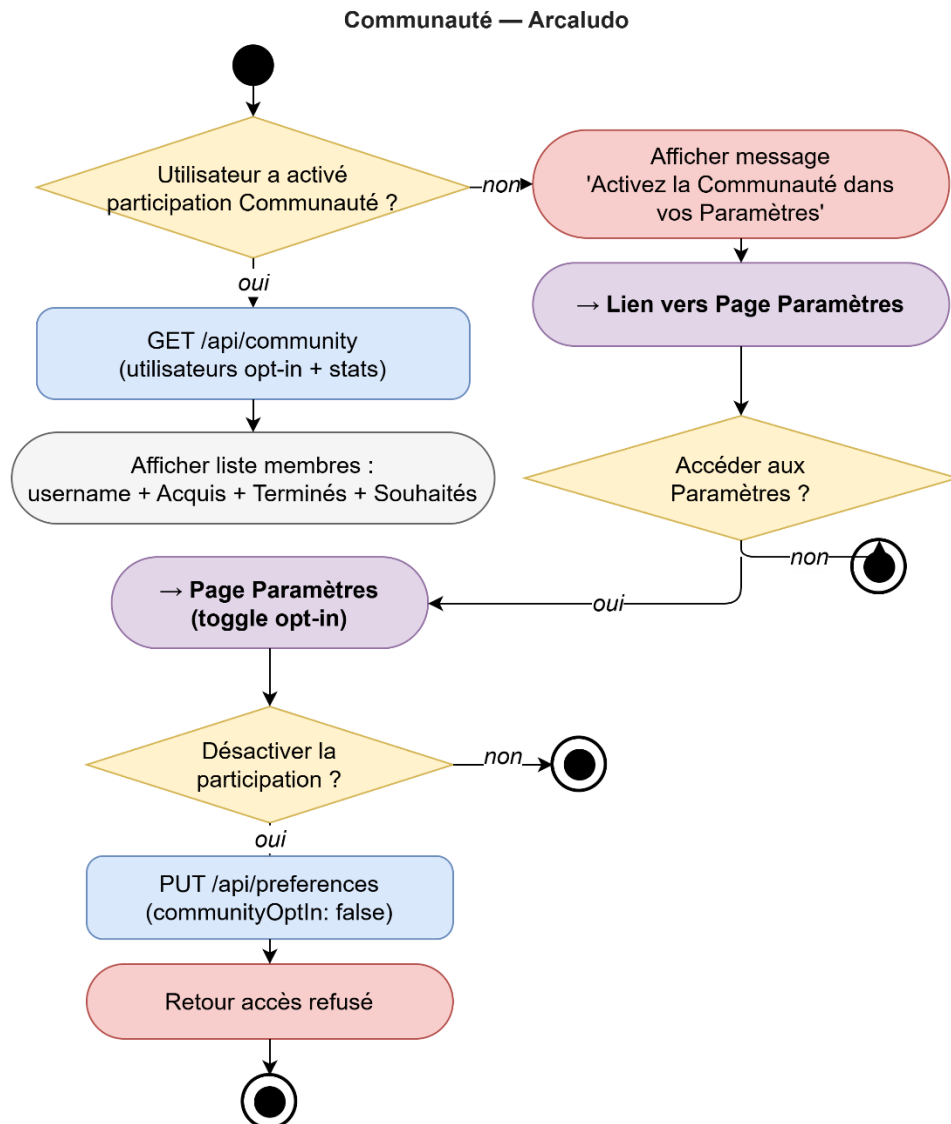
La Bibliothèque sert à chercher des jeux. Par défaut, elle montre les jeux populaires. L'utilisateur peut taper un nom, filtrer ou trier les résultats. Si un jeu est déjà dans la collection, une notification l'indique. Sinon, il peut l'ajouter ou le mettre en souhaits ou sur sa collection.

1.5.2.5. Page Détail du jeu



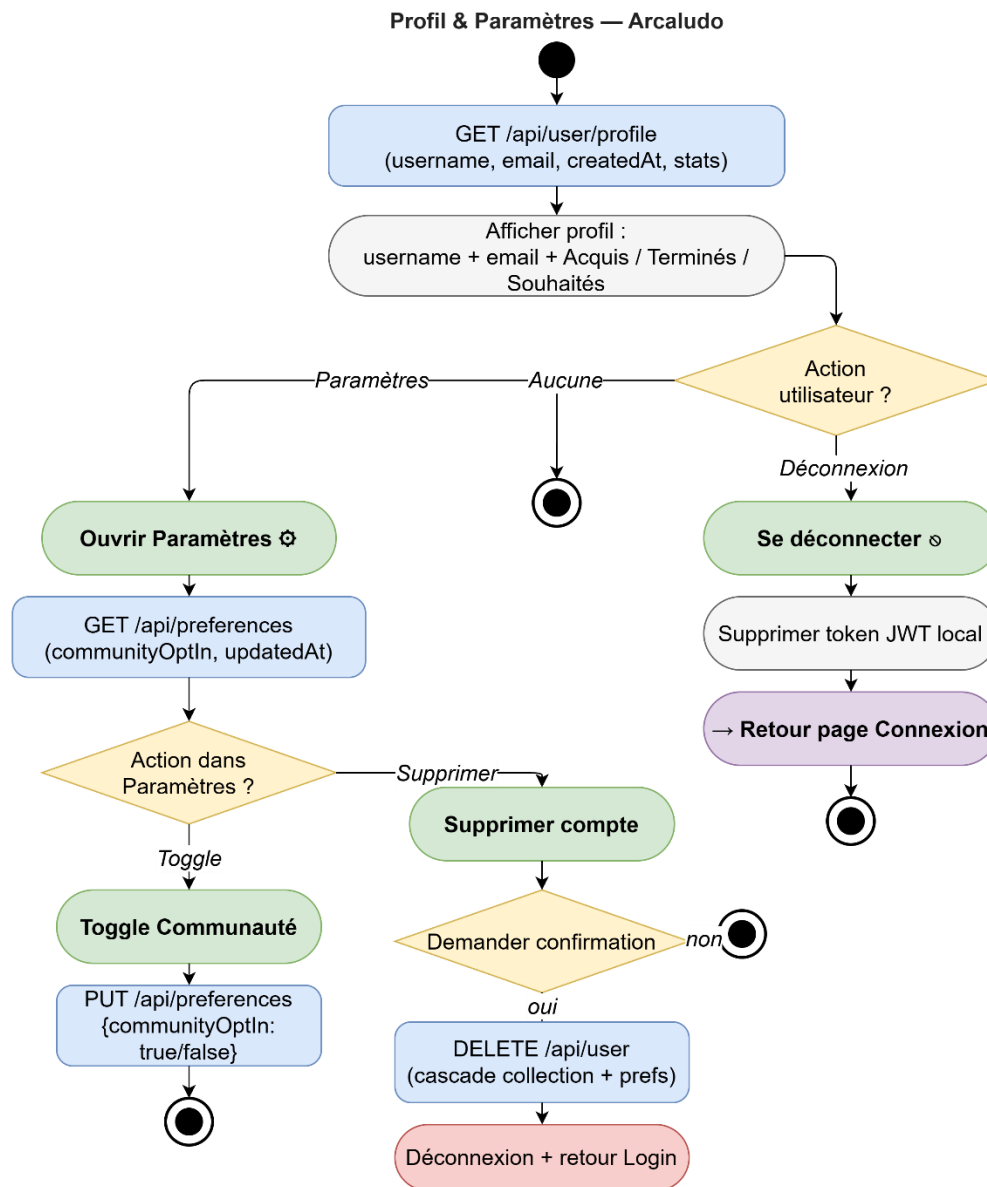
Ici s'affiche toutes les infos d'un jeu. Si le jeu est déjà dans la collection, les champs sont remplis. L'utilisateur peut choisir un statut, noter, commenter, ajouter du temps de jeu, enregistrer, ajouter ou retirer le jeu.

1.5.2.6. Page Communauté



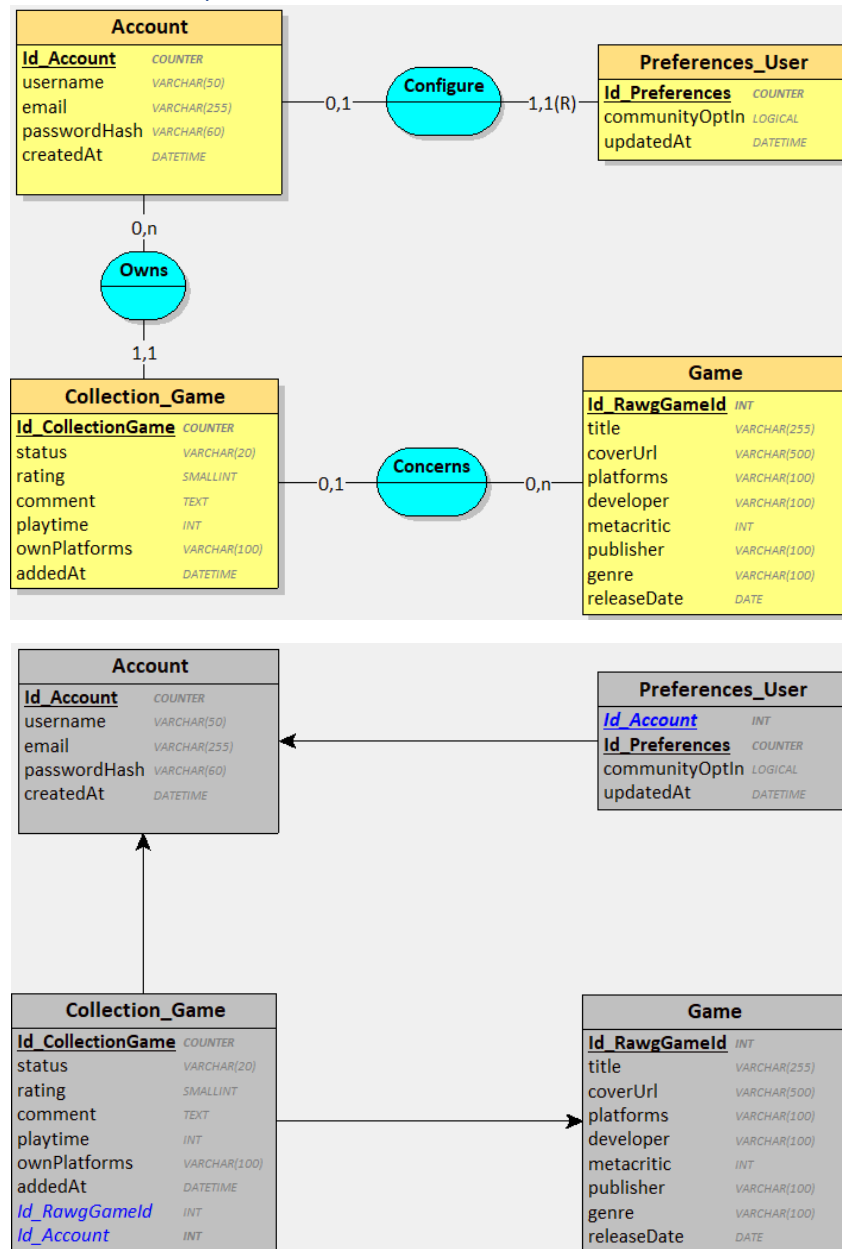
La Communauté n'est accessible que si l'utilisateur a activé l'option. S'il l'a fait, il voit les autres membres participants et leurs statistiques publiques. Il peut activer ou désactiver cette option dans les paramètres.

1.5.2.7. Page Profil



La page Profil montre les infos de l'utilisateur et ses statistiques. Depuis là, il peut aller aux paramètres ou se déconnecter. Dans les paramètres, il peut gérer sa participation à la communauté ou supprimer son compte

1.5.3. Modèle conceptuel de données



Le MCD d'Arcaludo a quatre entités. `t_user` contient les infos du compte (nom, email, mot de passe, date de création). Chaque utilisateur possède des préférences dans `t_preferences_user`, où se trouve surtout l'option pour activer ou non la participation à la communauté.

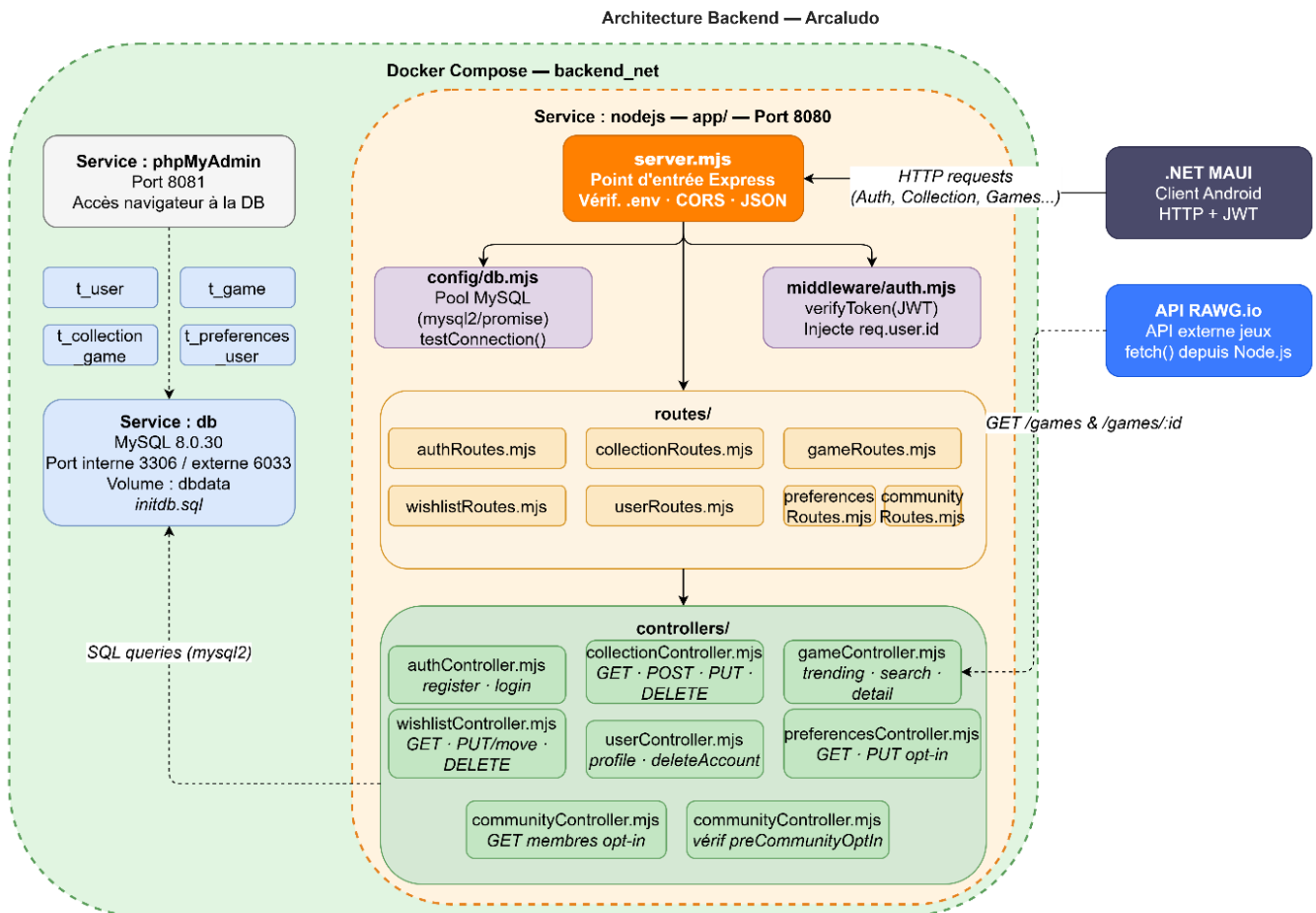
Un utilisateur peut avoir plusieurs jeux dans sa collection, gardée dans `t_collection_game`. Chaque entrée contient le statut du jeu (acquis, en cours, terminé, souhait), une note, un commentaire, le temps de jeu et les plateformes possédées. Si le compte est supprimé, toute la collection disparaît automatiquement.

Chaque entrée de collection peut être liée à un jeu du cache local, enregistré dans `t_game`. Ce cache contient les infos de base d'un jeu (titre, cover, genres, plateformes, etc.). Si un jeu est supprimé du cache, l'entrée de collection reste simplement sans l'information.

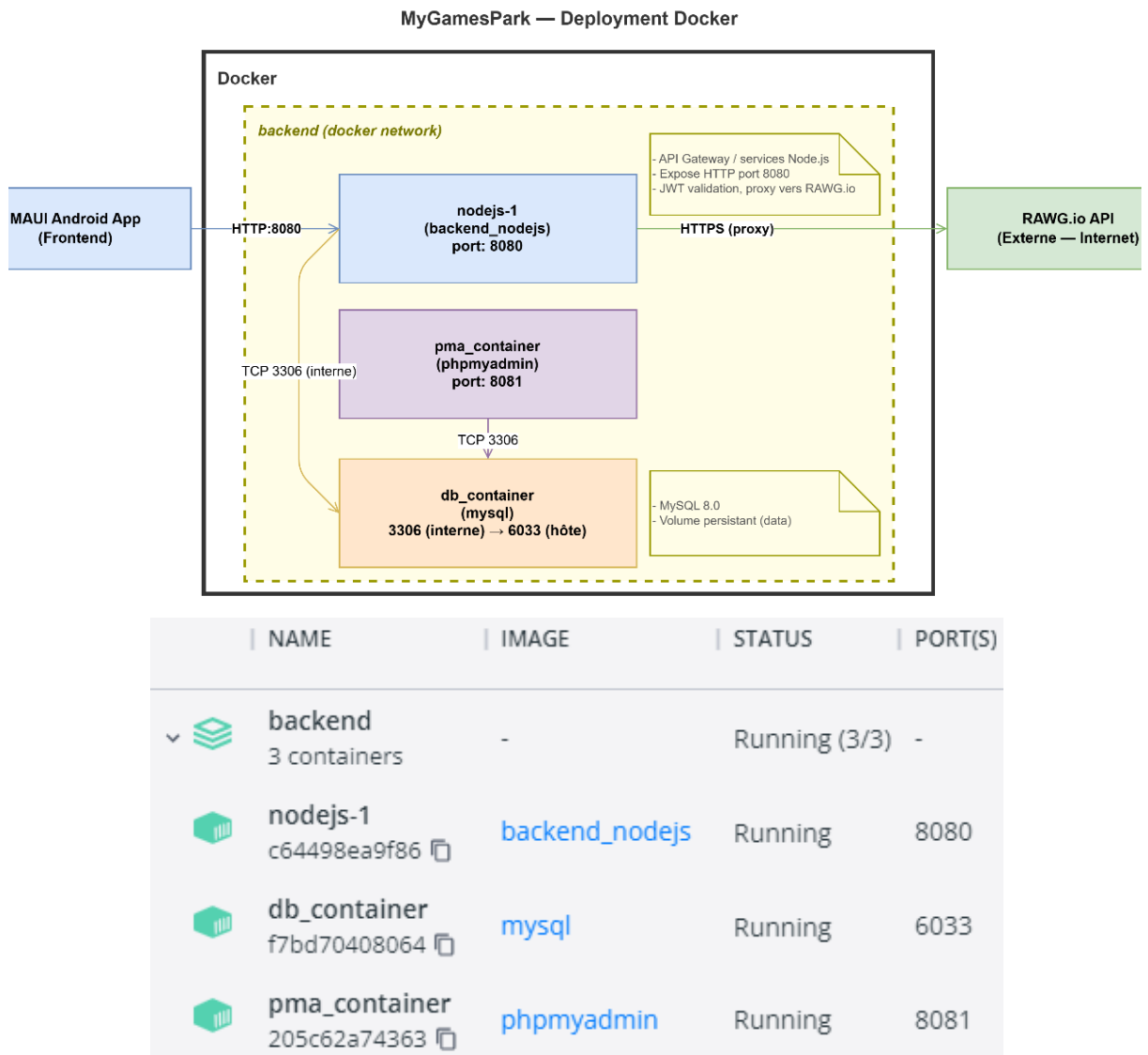
1.5.4. Cas d'utilisation

1.5.5. Classes

1.5.6. Architecture Backend

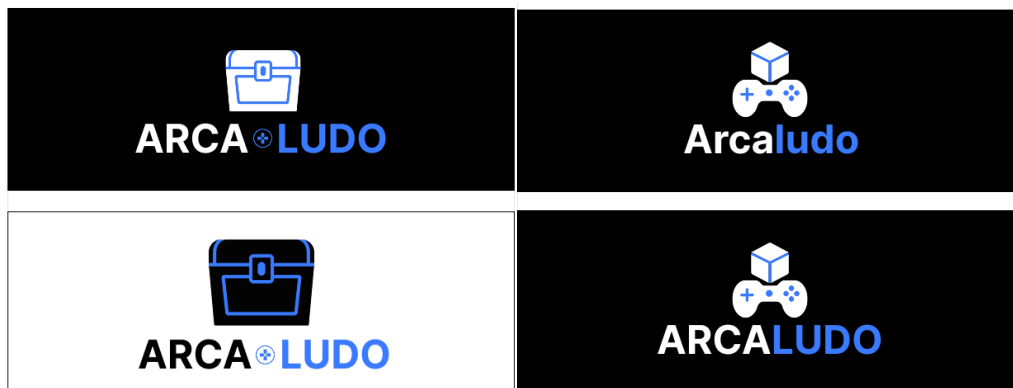


1.5.7. Docker



1.5.8. Maquettes

- Logo



La conception du logo est partie de plusieurs propositions initiales pour intégrer la couleur principale de l'application, le bleu, ainsi qu'un élément visuel représentant à la fois les jeux et l'idée de boîte ou de conteneur.

Pour sélectionner l'option finale, trois critères principaux ont été évalués : le contexte, la clarté et la fonction. Tout d'abord, la couleur de fond de l'application a été définie, le choix s'étant porté sur une teinte noire/sombre, car elle réduit la consommation d'énergie sur les écrans modernes et s'adapte naturellement au mode sombre des appareils mobiles.

Ensuite, différents symboles à représenter la collection de jeux ont été étudiée. D'abord un coffre, mais son image visuelle c'était complexe. C'est pourquoi j'ai choisi une icône composée d'une boîte et d'une manette, capable de représenter de manière minimaliste l'ensemble des toutes les consoles.

En ce qui concerne la typographie « Inter » du logo, la couleur bleue a été conservée comme élément distinctif, en écartant la séparation entre les mots afin de favoriser une identité plus reconnaissable. L'utilisation des minuscules a également été évitée, car elle créait un déséquilibre visuel. Finalement, le choix s'est porté sur un texte symétrique en majuscules, combinant le blanc et le bleu pour renforcer la lisibilité.

La proposition finale, validée et approuvée par le chef de projet, est la suivante :



- Maquettes – Pages

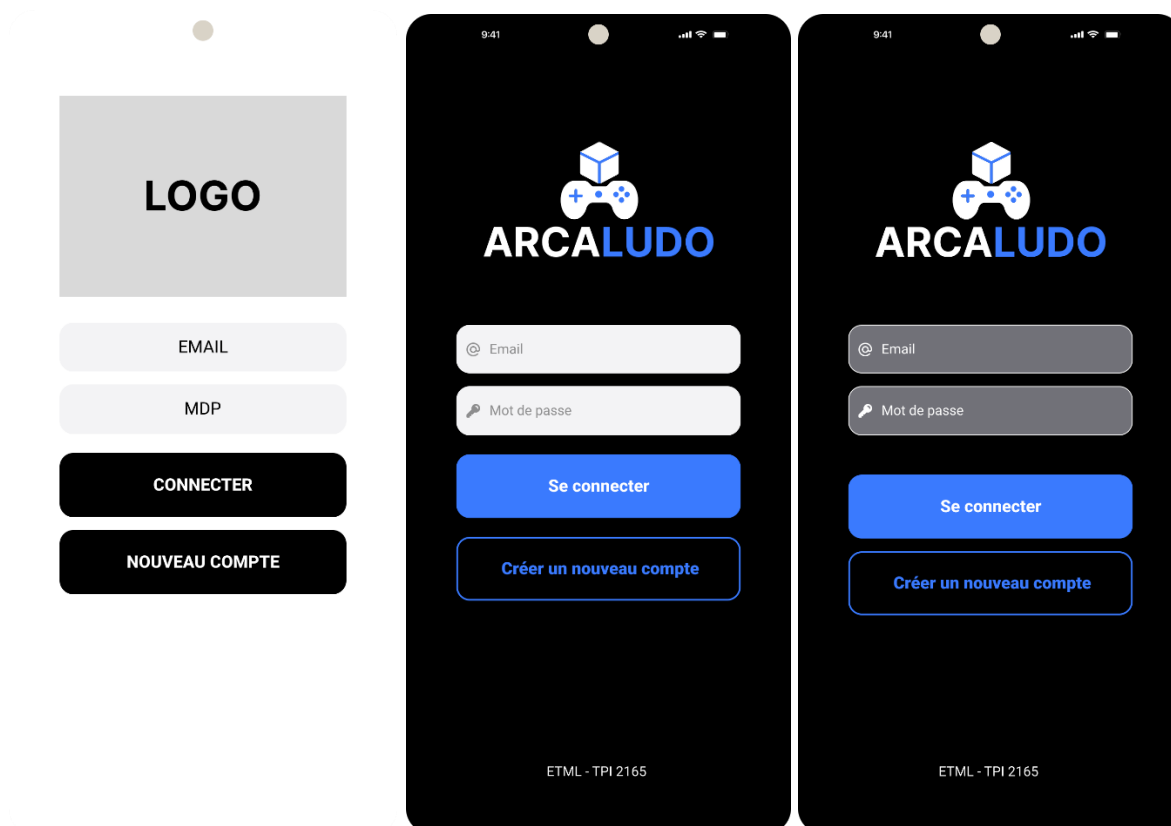
Les maquettes d'Arcaludo ont été créées dans Figma avec un design cohérent et inspiré avec autres applications mobile similaire déjà mentionné. Le fond utilise un noir profond pour réduire la fatigue visuelle. Le bleu vif sert de couleur principale pour les actions importantes et pour rester cohérent avec le logo. La navigation et les en-têtes utilisent un gris-violet foncé qui donne de la profondeur sans distraire. Les statuts des jeux sont différenciés avec des couleurs simples : jaune doré pour Acquis, bleu claire pour En cours, vert clair pour Terminés et rose vif pour Souhaits.

La typographie choisie est Inter, une police très lisible sur mobile, utilisée pour le texte régulier et en gras pour les titres.

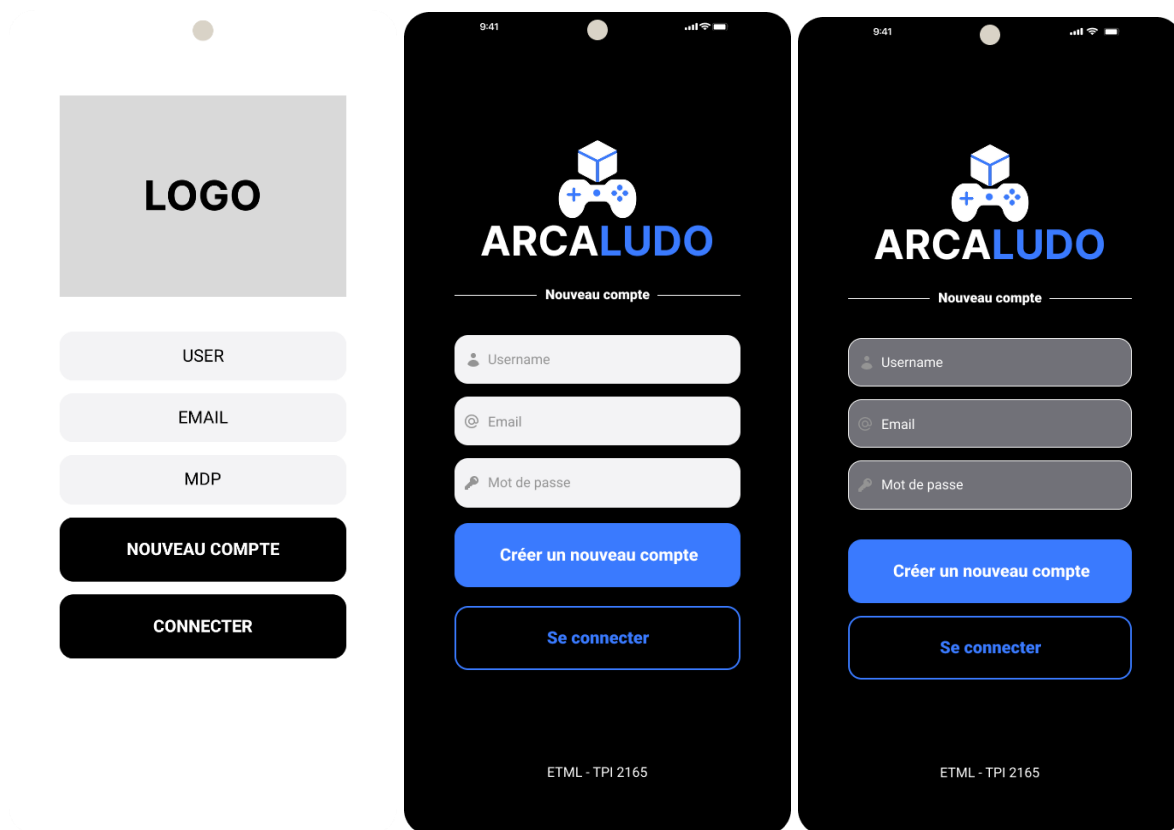
Les éléments visuels suivent une logique uniforme : les boutons principaux sont bleus et arrondis, les boutons secondaires sont en contour bleu, et les cartes d'information utilisent un gris très clair avec une bordure colorée selon le statut du jeu. La barre de navigation a cinq icônes faciles à reconnaître pour les pages de : Collection, Ajouter, Souhaits, Communauté, Profil. Tous les icones sont pris du Heroicons.

Voici les maquettes correspondant aux trois étapes du processus : « basse fidélité », « haute fidélité » et « haute fidélité » avec les remarques de mon chef de projet.

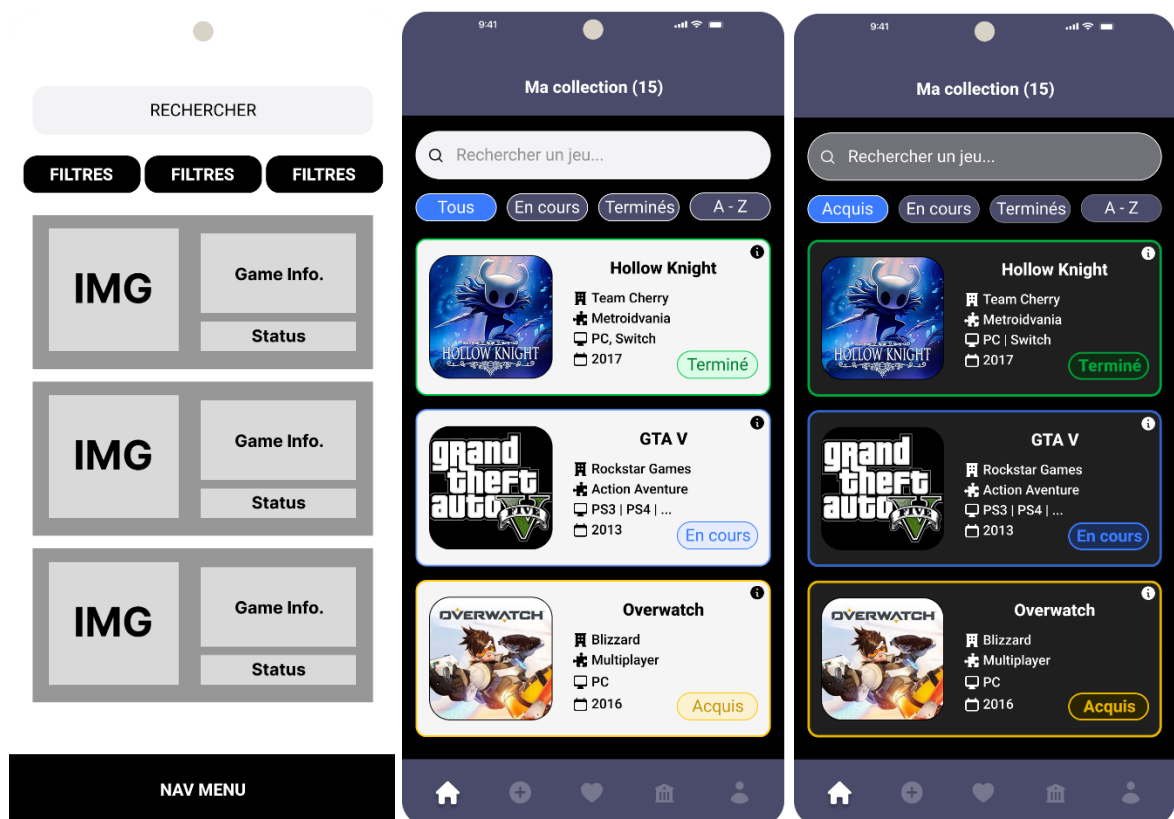
- Se connecter



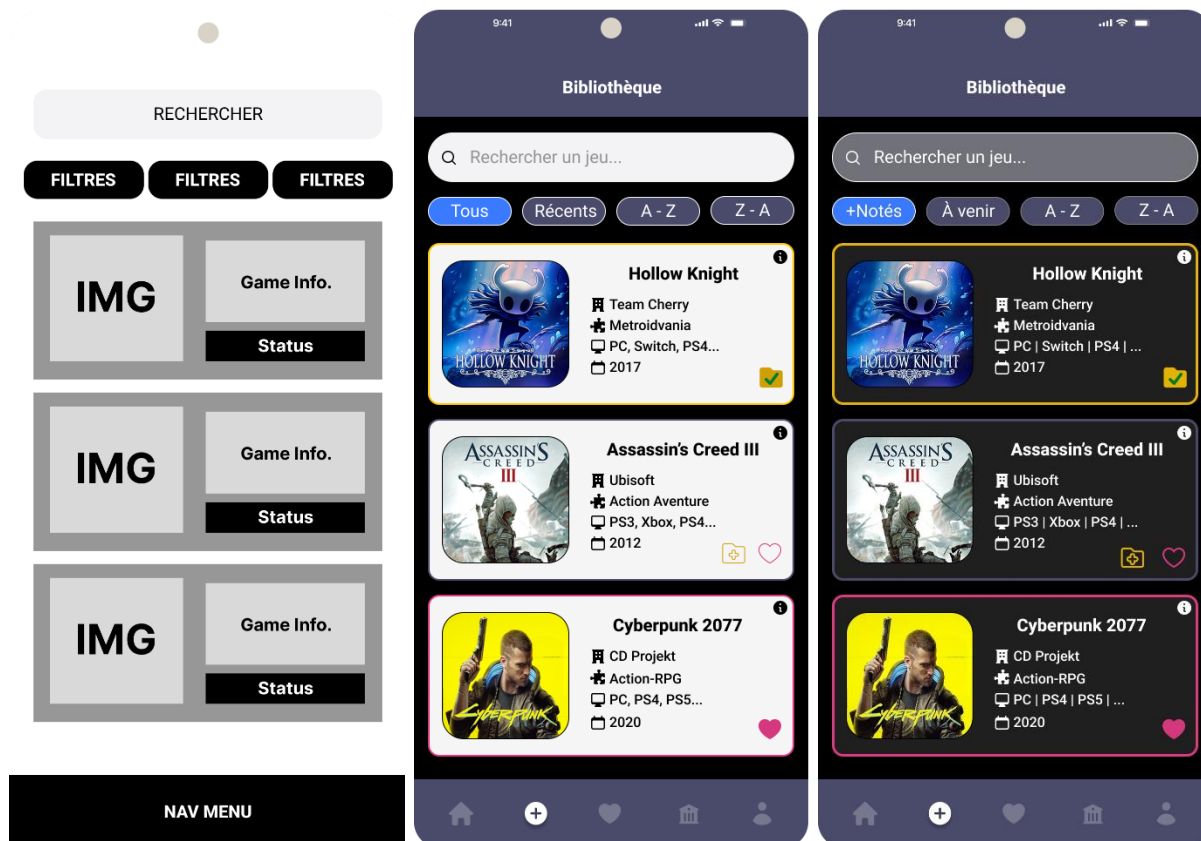
- Nouveau compte



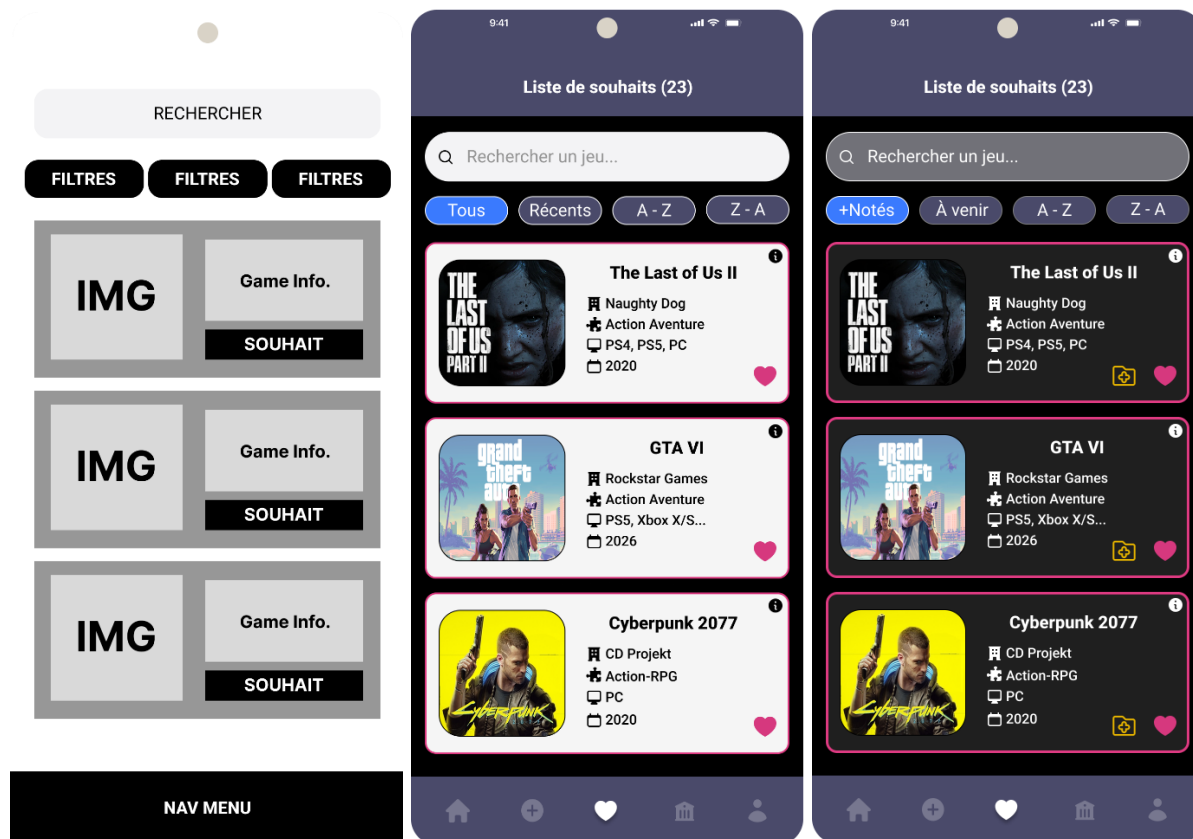
- Ma collection



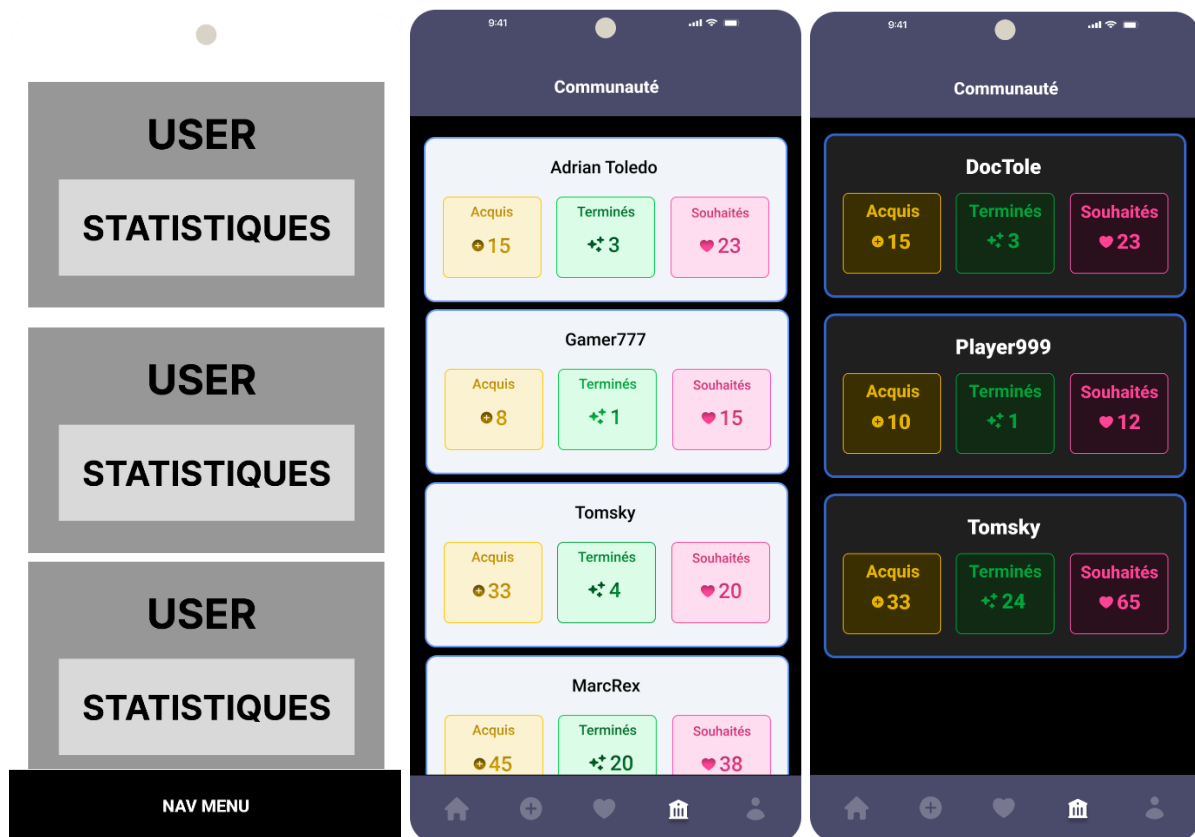
- Bibliothèque



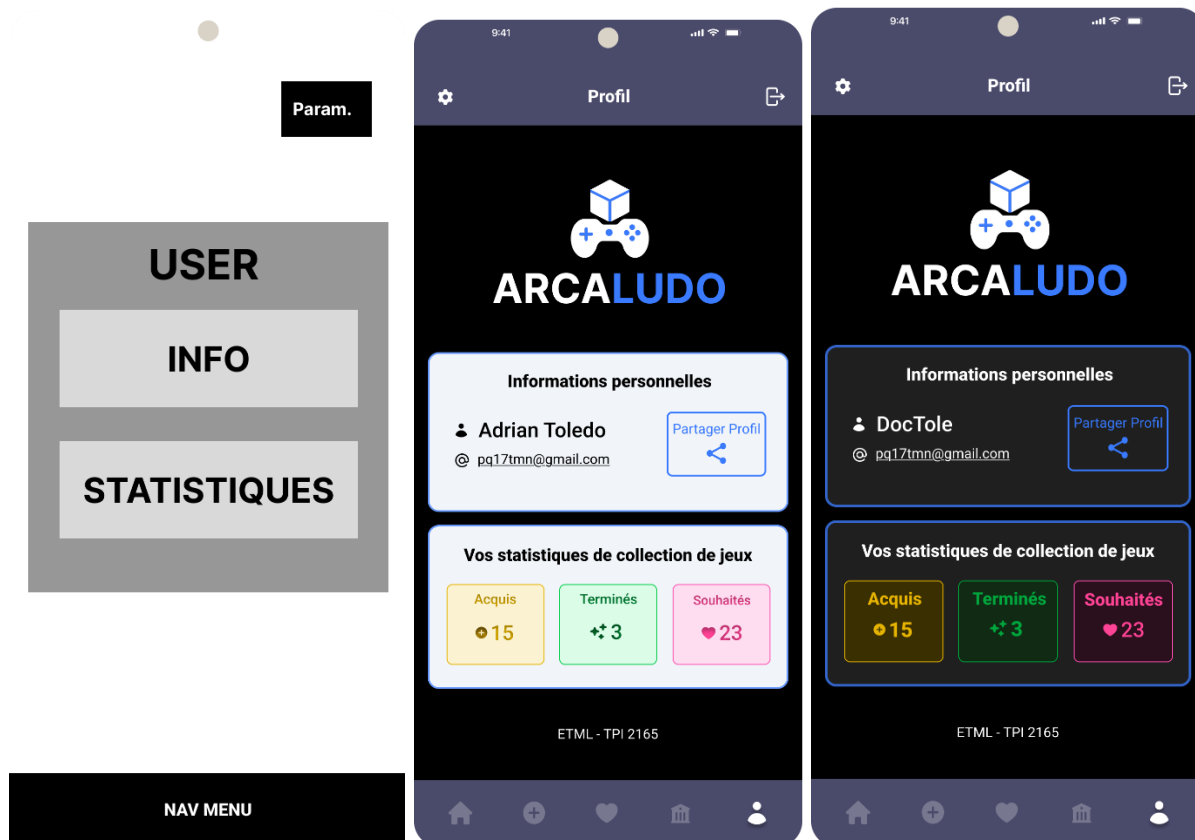
- Liste de souhaits



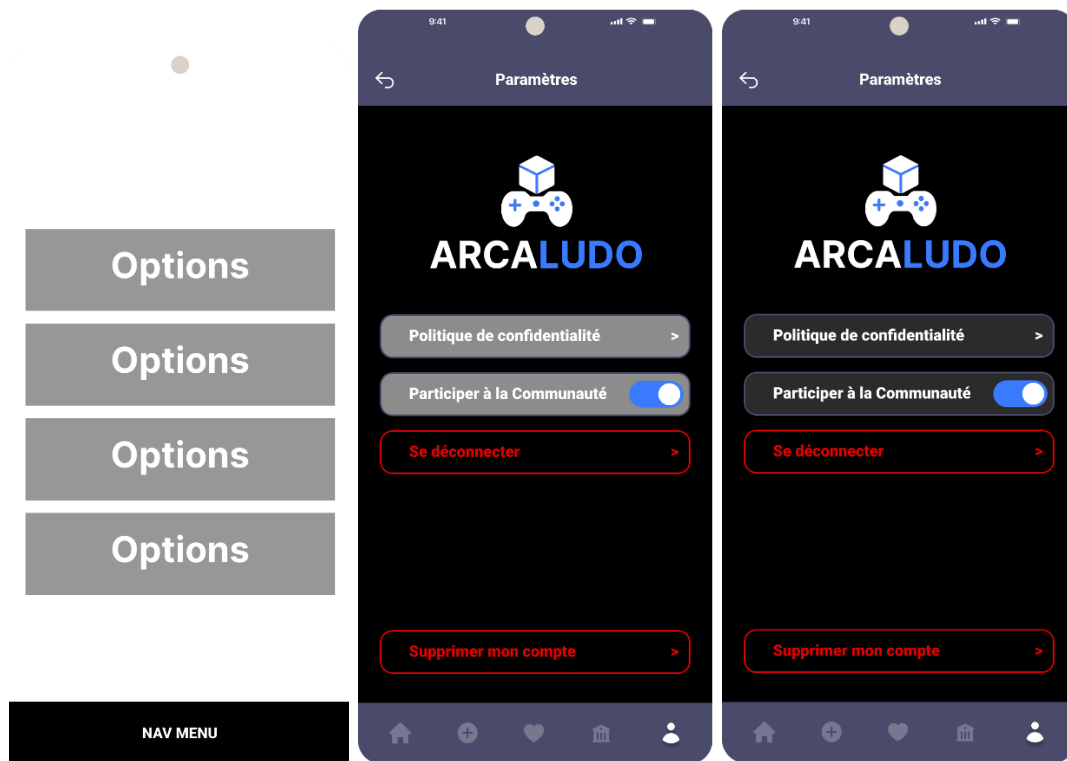
- Communauté



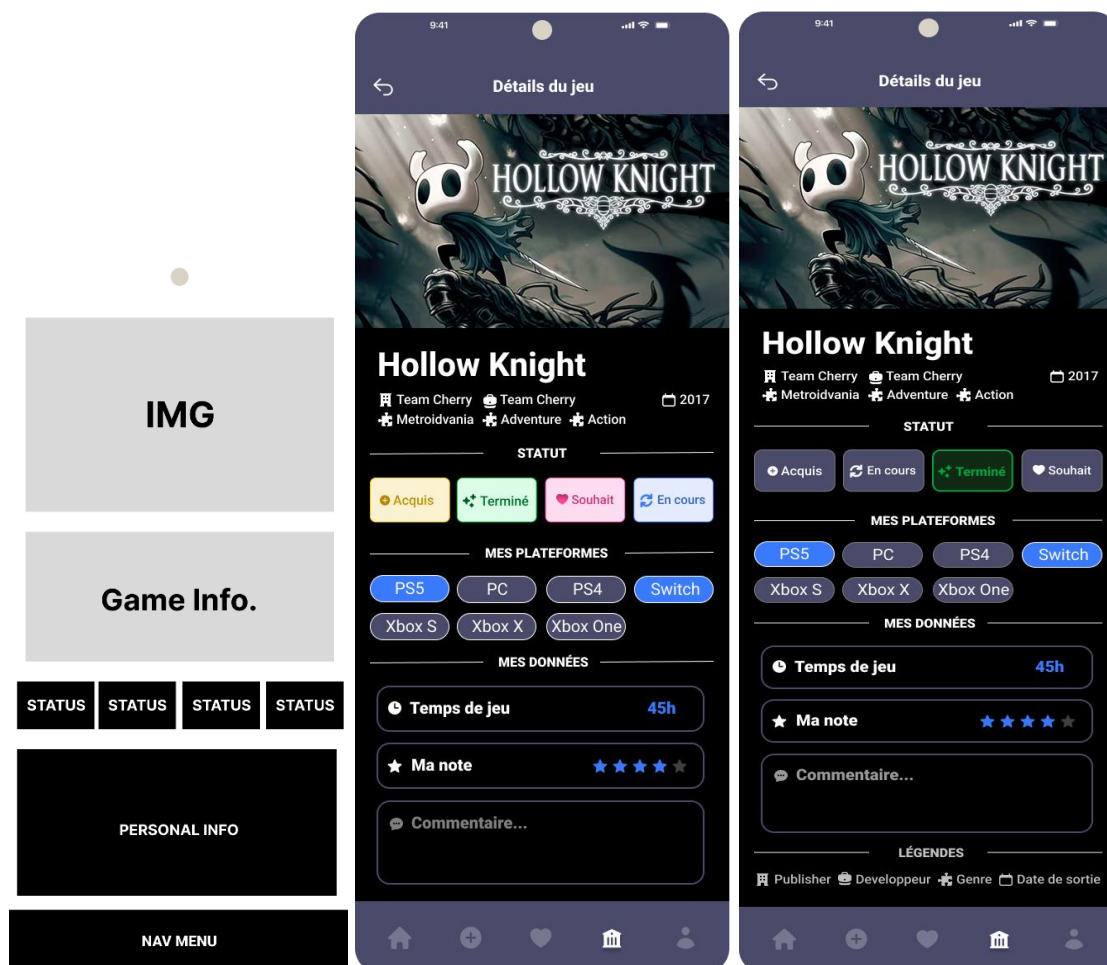
- Profil



- Paramètres



- Détails du jeu



1.5.9. Planification de tests

1. Authentification — public (sans token)			
Méthode	Endpoint	Description	Résultat attendu
POST	/api/auth/register	Créer un compte utilisateur	201 — Compte créé avec préférences par défaut
POST	/api/auth/login	Connexion et obtention du token JWT	200 — Token JWT retourné
2. Profil utilisateur			
Méthode	Endpoint	Description	Résultat attendu
GET	/api/user/profile	Afficher profil et statistiques	200 — username, email, acquis, terminés, souhaités
DELETE	/api/user	Supprimer le compte (effacement nLPD)	200 — Compte et données effacés en cascade
3. Préférences			
Méthode	Endpoint	Description	Résultat attendu
GET	/api/preferences	Lire le statut d'opt-in Communauté	200 — {communityOptIn: false}
PUT	/api/preferences	Activer la participation Communauté	200 — Confirmation mise à jour
4. Communauté			
Méthode	Endpoint	Description	Résultat attendu
GET	/api/community	Liste des membres opt-in avec stats	200 — Tableau username + statistiques
GET	/api/community	Accès refusé si opt-in désactivé	403 — Message d'accès refusé
5. Bibliothèque RAWG			
Méthode	Endpoint	Description	Résultat attendu

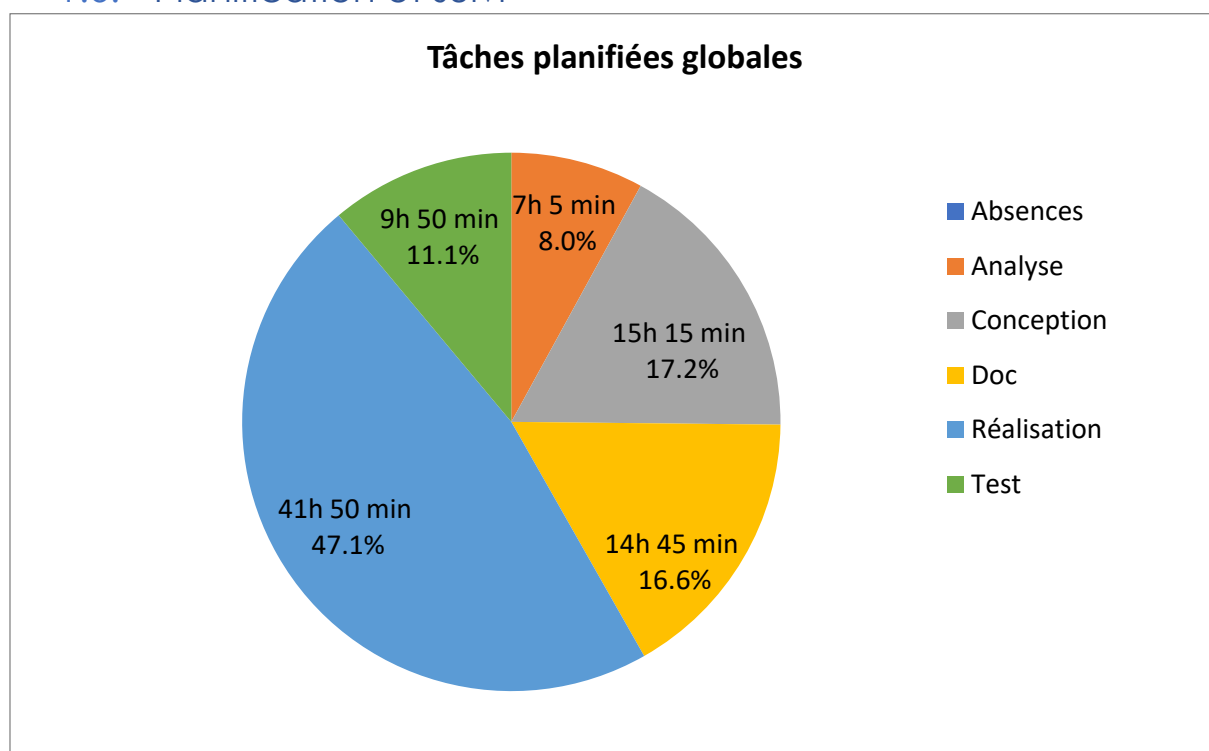
GET	/api/games/trending	Jeux tendances (tri par défaut +Notés)	200 — 20 jeux, developer null (limite RAWG)
GET	/api/games/trending?sort=az	Tri alphabétique A-Z	200 — Titres latins uniquement
GET	/api/games/trending?sort=recent	Jeux récents déjà sortis	200 — Aucun jeu futur
GET	/api/games/trending?sort=avenir	Jeux à venir triés par date de sortie	200 — Dates futures, plus proches en premier
GET	/api/games/search?q=hollow+knight	Recherche par titre	200 — Résultats + flag inCollection par jeu
GET	/api/games/3498	Détail GTA V avec developer et publisher	200 — developer et publisher remplis
6. Collection			
Méthode	Endpoint	Description	Résultat attendu
POST	/api/collection	Ajouter GTA V avec statut playing	201 — colld retourné
GET	/api/collection	Liste complète (wishlist exclue)	200 — JOIN t_game, triés par date ajout
GET	/api/collection?status=playing	Filtre par statut playing	200 — Seulement les jeux en cours
GET	/api/collection?q=grand	Recherche par titre dans la collection	200 — Filtre sur gamTitle
GET	/api/collection?sort=az	Tri alphabétique A-Z dans la collection	200 — Ordre gamTitle ASC
PUT	/api/collection/:colld	Modifier statut, note, commentaire, temps	200 — Champs mis à jour
DELETE	/api/collection/:colld	Supprimer un jeu de la collection	200 — Jeu retiré
POST	/api/collection	Ajouter Witcher 3 en liste de souhaits	201 — Ajouté avec status wishlist
7. Liste de souhaits			
Méthode	Endpoint	Description	Résultat attendu

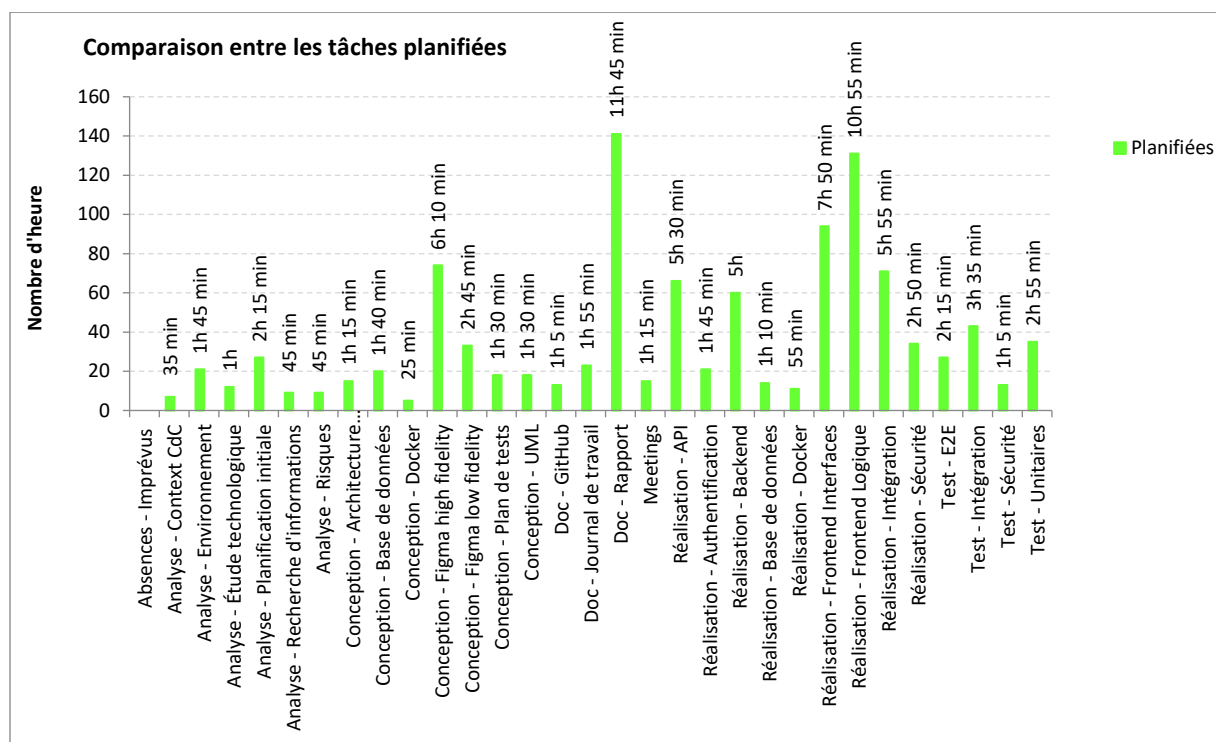
GET	/api/wishlist	Liste souhaits triés par date ajout	200 — Plus récents en premier
GET	/api/wishlist?sort=notes	Tri par note Metacritic	200 — gamMetacritic décroissant
GET	/api/wishlist?q=witcher	Recherche par titre dans la wishlist	200 — Filtre sur gamTitle
PUT	/api/wishlist/:colld/move	Déplacer vers la collection	200 — Statut mis à jour
DELETE	/api/wishlist/:colld	Retirer un jeu des souhaits	200 — Entrée supprimée

1.5.10. Versions des systèmes et outils

Composant	Outil	Version
Système d'exploitation	Windows 11 et Android 13	
Framework Frontend	.NET Maui	9.0
Environnement dev.	Visual Studio Community 2022	17.8
Runtime Backend	Node.js	20.11.0
Base de données	MySQL	8.0
Gestionnaire de paquets	npm	10.2.4
Conteneurisation	Docker Desktop	4.27.0

1.6. Planification et suivi





- 1.7. Réalisation
- 1.8. Tests et validation
- 1.9. Livraison
- 1.10. Conclusion et bilan personnel

- 2. Documentation du produit
 - 2.1. Prérequis et environnement cible
 - 2.2. Installation et déploiement
 - 2.3. Guide d'utilisation
 - 2.4. Exploitation et maintenance
 - 2.5. Architecture technique

- 3. Résumé
 - 3.1. Contexte et mandat
 - 3.2. Solution réalisée
 - 3.3. Résultats principaux
 - 3.4. Limites et suites

- 4. Annexes
 - 4.1. Journal de travail
 - 4.2. Bibliographie et références

 - 4.3. Liste des livrables remis

Légende : □ Action UI • Action Backend (BD) ◆ Décision